Module Code
COM00018I

DEPARTMENT OF COMPUTER SCIENCE

Systems & Devices 3: Advanced Computer Systems

Laboratory Practicals

Introductory Guide notes

Version 2022-01-12

Experimental Pipeline Optimisation

1. SYNOPSYS OF ACTIVITIES :

This year's SYS3 lab sessions are based upon a set of formative activities (not formative assessment). These activities are intended to encourage you to investigate various problems relating to processor architecture, in particular the complexities of pipelines in scalar and superscalar processors.

In the following lab sessions you will be attempting to develop a series of solutions to problems relating to pipeline hazards in a hypothetical scalar/superscalar processor architecture. This will be achieved with the assistance of a small function library containing the ***PipelineClass*** *c++* class, which provides the basic capabilities of generating and representing pipeline stages for instruction sequences, along with some basic convenience-functions. Documentation for the library is provided as explained in the appendix to this assessment document. An example of a simple pipeline diagram generated from the tool is given in **fig.1.**.

You will be required to understand the ***PipelineClass*** c++ class capabilities, investigate pipeline hazards and solutions, potentially including the option of additional research of these concepts if desired[1], and to write C/C++[2] programs to achieve various objectives relating to correcting pipeline hazards that arise when specific test cases are fed into the pipeline generator. You will devise solutions that eliminate the pipeline hazards and document your results according to the assessment specification given.

Test cases will be provided for each sub-task A, B, and C. More detail will be provided in briefing lecture(s) at the start of term. This document contains further sections: (2) list of abbreviations, (3) assessment specification and marking scheme, and (4) details of appendices as referred to above.

Lab conduct:
In the lab we would encourage you to attempt your own solutions hands-on, in order to maximise the learning experience, particularly in the case of the first two activities.

However you should feel able to speak to students sitting next to you, or in small groups of 2 or 3 from time to time, to discuss ideas - the aim is to learn as individuals but open discussion of ideas and questions is a useful part of that process.

The tasks suggest typical periods of study (e.g. weeks 2 to 3). You should try to spend time on each sub-topic rather than doing everything in activity 1 and 2 and none in activity 3, for example.

---

[1] For example, additional reading, online tutorials, research literature, are all sensible possibilities to gain further insight into problems and possible solutions beyond the most basic cases.

[2] There is very limited C++ language syntax used in these labs. We will use a C++ compiler, gcc, however much of your code will be the same as standard C, (unless of course you wish to use C++ more comprehensively - this is allowed).

```
---- PIPELINE DIAGRAM----
000:      LDR.       R1.       R5.           >> [I=000]  [--IF--][--ID--][RR..R1][--OF--][WB..R5]
001:      LDR.       R5.       R6.           >> [I=001]  [  --  ][--IF--][--ID--][RR..R5][--OF--][WB..R6]
002:      LDI.       R3.      992.           >> [I=002]  [  --  ][  --  ][--IF--][--ID--][--OF--][WB..R3]
003:      LDI.       R2.      221.           >> [I=003]  [  --  ][  --  ][  --  ][--IF--][--ID--][--OF--][WB..R2]
004:      ADD.       R2.       R1.       R3  >> [I=004]  [  --  ][  --  ][  --  ][  --  ][--IF--][--ID--][RR..R2][RR..R1][--IA--][WB..R3]
005:      LDM.       R1.    $9983.           >> [I=005]  [  --  ][  --  ][  --  ][  --  ][  --  ][--IF--][--ID--][--OF--][--DF--][WB..R1]
006:      LDR.       R5.       R1.           >> [I=006]  [  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][--IF--][--ID--][RR..R5][--OF--][WB..R1]
007:      NOP.         .         .           >> [I=007]  [  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][--IF--][--ID--]
008:      ADD.       R9.       R8.       R3  >> [I=008]  [  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][--IF--][--ID--][RR..R9][RR..R8][--IA--][WB..R3]
009:      LDM.       R3.    $9983.           >> [I=009]  [  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][--IF--][--ID--][--OF--][--DF--][WB..R3]
010:      NOP.         .         .           >> [I=010]  [  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][  --  ][--IF--][--ID--]

                                                         [C-0000][C-0001][C-0002][C-0003][C-0004][C-0005][C-0006][C-0007][C-0008][C-0009][C-0010][C-0011]

-- [ MEMORY HAZARDS ] ----------------
 Memory Hazard(s) at Instruction 3 Cycle 3 [--IF--]
 Memory Hazard(s) at Instruction 2 Cycle 4 [--OF--]
 Memory Hazard(s) at Instruction 4 Cycle 4 [--IF--]
 Memory Hazard(s) at Instruction 5 Cycle 5 [--IF--]
 Memory Hazard(s) at Instruction 7 Cycle 7 [--IF--]
 Memory Hazard(s) at Instruction 8 Cycle 8 [--IF--]
 Memory Hazard(s) at Instruction 9 Cycle 9 [--IF--]

-- [ REGISTER HAZARDS ] ----------------
 RAW Hazard ? --  RR.5(i1,c3)) WB.5,(i0,c4)
 RAW Hazard ? --  RR.2(i4,c6)) WB.2,(i3,c6)
 WAW Hazard ? --  RR.3(i9,c13)) WB.3,(i8,c13)

-- [ STRUCTURAL HAZARDS ] ----------------
 WRITE PORTS Exceeded at c05 (2 used vs 1 available)
 WRITE PORTS Exceeded at c09 (2 used vs 1 available)
 WRITE PORTS Exceeded at c13 (2 used vs 1 available)
```

Fig.1. Example pipeline diagram generated by helper tool.

2. ABBREVIATIONS USED IN THE LAB SHEETS AND DOCUMENT

IF	Instruction Fetch
ID	Instruction Decode
OF	Operand Fetch
DF	Data Fetch
DS	Data Write
IA	Integer ALU execute cycle
FA	Floating point ALU execute cycle
SA	Shift unit (SHU) execute cycle
ALU	Arithmetic Logic Unit
IALU	Integer ALU
FPALU	Floating Point ALU
SHU	Shift/Binary Bit Manipulation ALU
IW	Issue Width
RR	Register Read
WB	Register Write Back

3. APPENDICES - Notes on Additional Documentation.

The following items will be released in week 2 as part of preliminary formative laboratory exercises.

1. **Instruction set and definitions, and CPU architecture description.**
   A description and example pipeline sequence will be provided for each instruction.
   Example:

LDI Rx, nn

Description: Load register X with immediate data value nn
RTL: Rx ← nn
Microsequence: [--IF--][--ID--][--OF--][WB..Rx]

---

2. **PipelineClass function reference**
   Definitions for all pipeline class helper functions and data structures will be provided.

   Example:

---

void InsertOp(int i,char * s1,char*s2,char*s3,char*s4);
Inserts an operation into the code list at instruction line i, consisting of opcode, operX, operY, OperZ

---

3. **Test cases**
   Test Assembler code sequences will be provided for each task.

   You can also create your own test cases.

   Example:

```
LDR     R1,     R5
LDR     R5,     R6
LDI     R3,     992
LDI     R2,     221
IADD    R2,     R1,R3
LDM     R1,     $9983
END
```

---

4. **Briefing materials**
   Lecture session(s) and appropriate slides will be provided at the start of term. Further discussions about some pipeline concepts will be incorporated in the ongoing lecture material to reinforce what you are learning in the labs through practical investigations.